Тесты проводились при помощи postman

Запрос

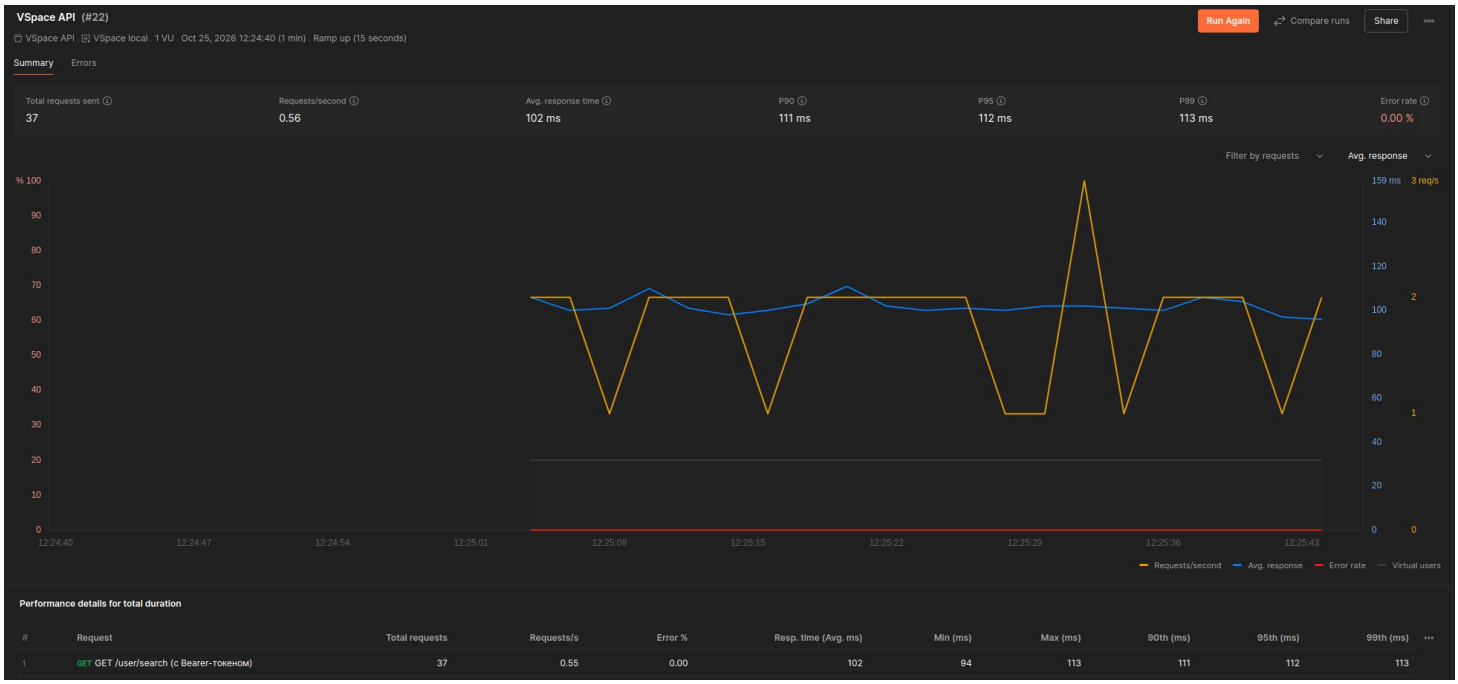
```
SELECT
    id,
    first_name,
    second_name,
    birthdate,
    biography,
    city
FROM users
WHERE LOWER(first_name) LIKE LOWER('Але%') ESCAPE '!'
  AND LOWER(second_name) LIKE LOWER('Ива%') ESCAPE '!'
ORDER BY id ASC;
```

Без использования индексов

План запроса

```
#  1 QUERY PLAN
1  Gather Merge  (cost=40308.70..40311.03 rows=20 width=98)
2    Workers Planned: 2
3    ->  Sort  (cost=39308.67..39308.70 rows=10 width=98)
4          Sort Key: id
5          ->  Parallel Seq Scan on users  (cost=0.00..39308.51 rows=10 width=98)
6                Filter: ((lower((first_name)::text) ~~ 'але%'::text) AND (lower((second_name)::text) ~~ 'ива%'::text))
```

1 VU


VSpace API (#22)
VSpace API VSpace local 1 VU Oct 25, 2026 12:24:40 (1 min) Ramp up (15 seconds)
Run Again
Compare runs
Share
Summary
Errors
Total requests sent
37
Requests/second
0.56
Avg. response time
102 ms
P90
111 ms
P95
112 ms
P99
113 ms
Error rate
0.00 %
Filter by requests
Avg. response
159 ms
3 req/s
Requests/second
Avg. response
Error rate
Virtual users
Performance details for total duration
Request Total requests Requests/s Error % Resp. time (Avg. ms) Min (ms) Max (ms) 90th (ms) 95th (ms) 99th (ms)
1 GET GET /user/search (с Bearer-токеном) 37 0.55 0.00 102 94 113 111 112 113

10 VU


VSpace API (#23)
VSpace API VSpace local 10 VU Oct 25, 2026 12:28:17 (1 min) Ramp up (15 seconds)
Run Again
Compare runs
Share
Summary
Errors
Total requests sent
381
Requests/second
5.73
Avg. response time
112 ms
P90
124 ms
P95
131 ms
P99
166 ms
Error rate
0.00 %
Filter by requests
Avg. response
178 ms
25 req/s
Oct 25, 2026 12:28:40 - 12:28:42
Requests/second 9
Avg. response 111 ms
Error rate 0%
Virtual users 5 VU
Requests/second
Avg. response
Error rate
Virtual users
Performance details for total duration
Request Total requests Requests/s Error % Resp. time (Avg. ms) Min (ms) Max (ms) 90th (ms) 95th (ms) 99th (ms)
1 GET GET /user/search (с Bearer-токеном) 381 5.67 0.00 112 93 264 124 131 166

100 VU



1000 VU



Для ускорения был выбран покрывающий индекс

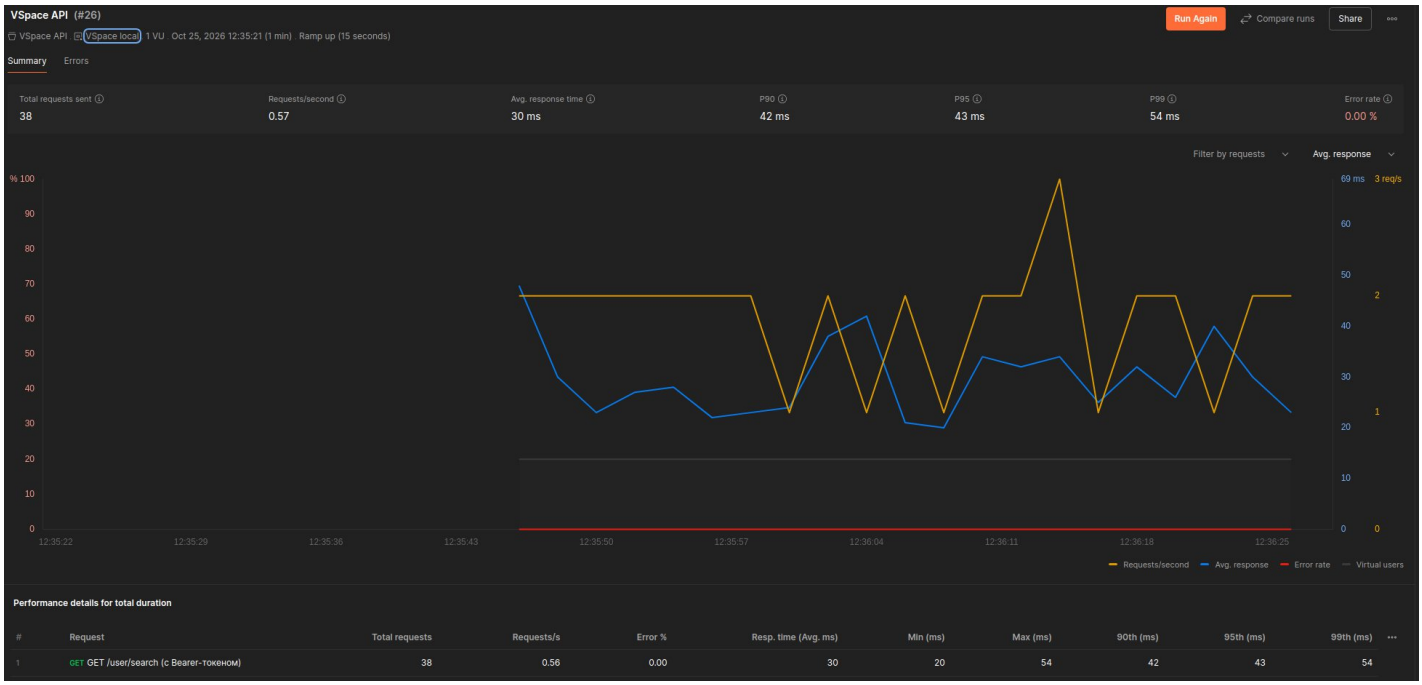
```
CREATE INDEX idx_users_lower_name_and_id_desc
ON users (
    (LOWER(first_name)) text_pattern_ops,
    (LOWER(second_name)) text_pattern_ops,
    id DESC
);
```

С использованием индексов

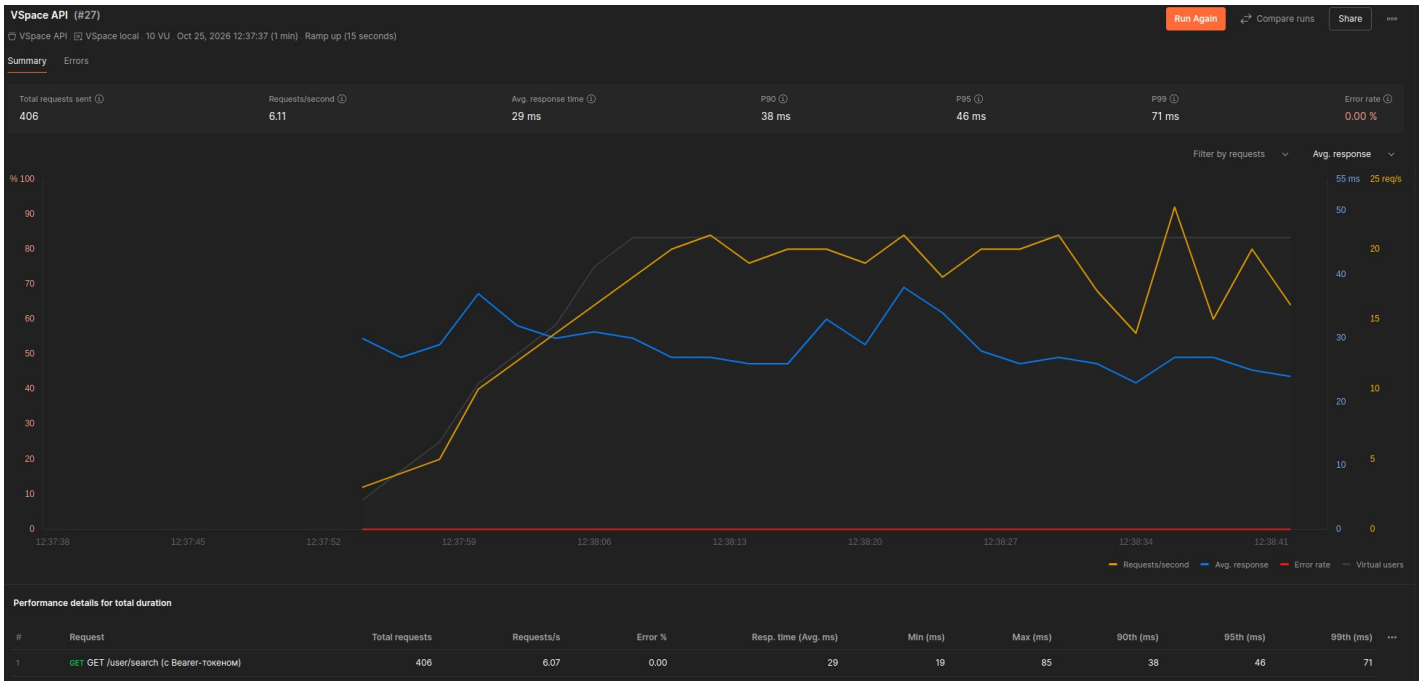
План запроса

```
# 1 QUERY PLAN
1 Sort  (cost=344.51..344.57 rows=25 width=98)
2   Sort Key: id
3   ->  Index Scan using idx_users_lower_name_and_id_desc on users  (cost=0.42..343.93 rows=25 width=98)
4         Index Cond: ((lower((first_name)::text) ~>=~ 'але'::text) AND (lower((first_name)::text) ~<~ 'алж'::text) AND (lower((second_name)::text) ~>=~ 'ива'::text) AND (lower((second_name)::text) ~<~ 'ивб'::text))
5         Filter: ((lower((first_name)::text) ~~ 'але%'::text) AND (lower((second_name)::text) ~~ 'ива%'::text))
```

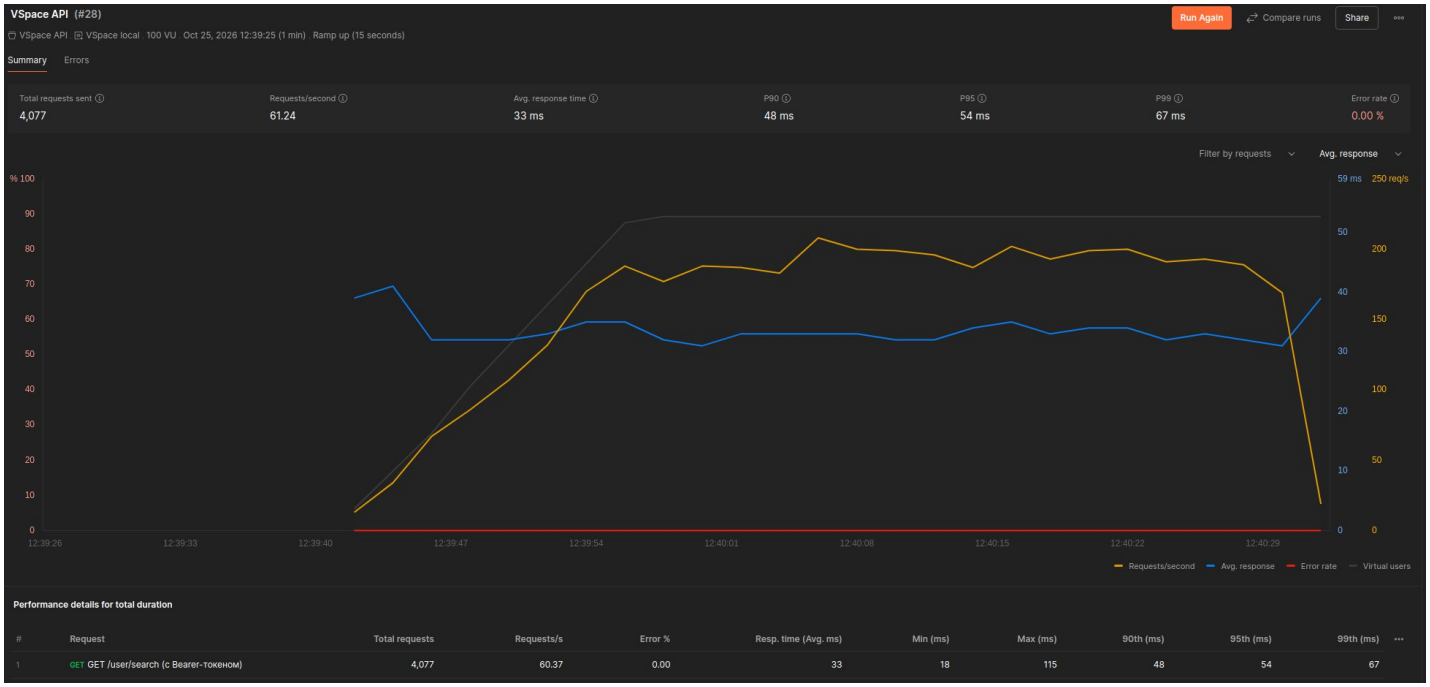
1 VU



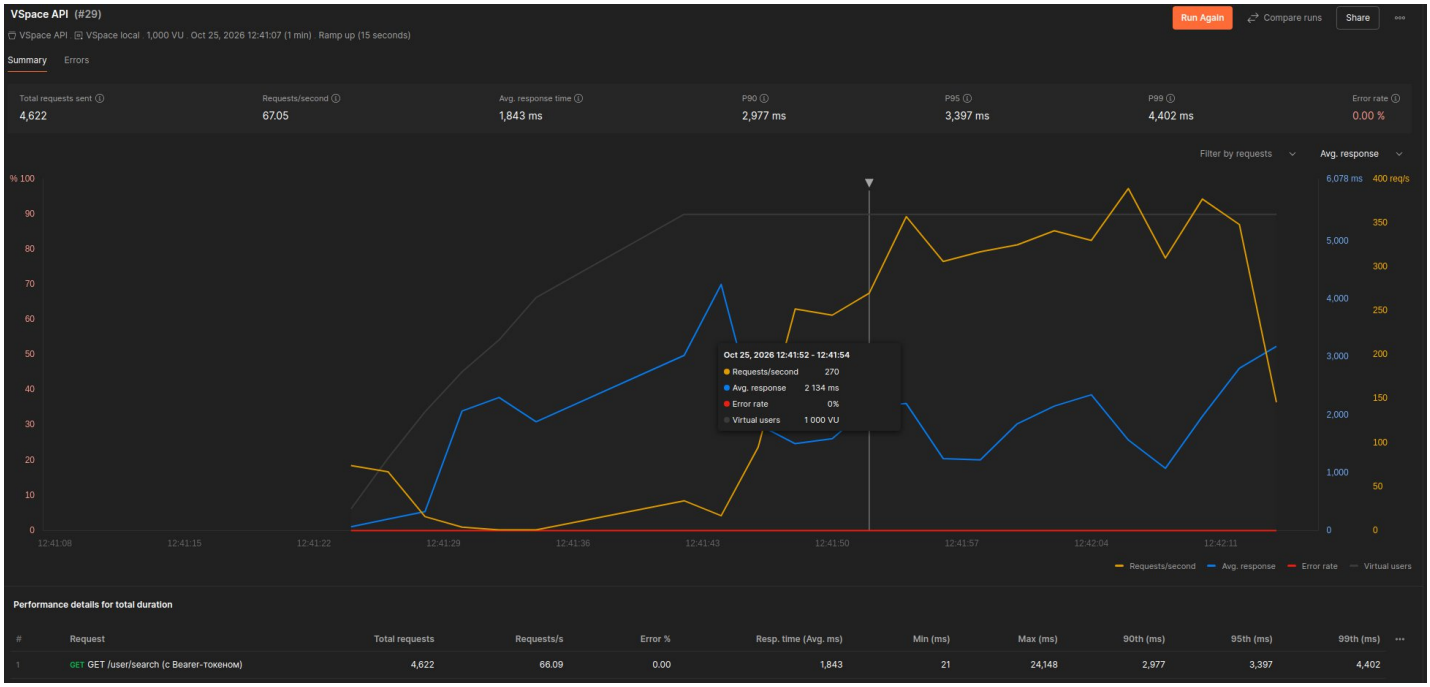
10 VU


VSpace API (#27)
Run Again
Compare runs
Share
VSpace API VSpace local 10 VU Oct 25, 2026 12:37:37 (1 min) Ramp up (15 seconds)
Summary
Errors
Total requests sent
406
Requests/second
6.11
Avg. response time
29 ms
P90
38 ms
P95
46 ms
P99
71 ms
Error rate
0.00 %
Filter by requests
Avg. response
Requests/second
Avg. response
Error rate
Virtual users
Performance details for total duration
Request Total requests Requests/s Error % Resp. time (Avg. ms) Min (ms) Max (ms) 90th (ms) 95th (ms) 99th (ms)
1 GET GET /user/search (с Bearer-токеном) 406 6.07 0.00 29 19 85 38 46 71

100 VU


VSpace API (#28)
Run Again
Compare runs
Share
VSpace API VSpace local 100 VU Oct 25, 2026 12:39:25 (1 min) Ramp up (15 seconds)
Summary
Errors
Total requests sent
4,077
Requests/second
61.24
Avg. response time
33 ms
P90
48 ms
P95
54 ms
P99
67 ms
Error rate
0.00 %
Filter by requests
Avg. response
Requests/second
Avg. response
Error rate
Virtual users
Performance details for total duration
Request Total requests Requests/s Error % Resp. time (Avg. ms) Min (ms) Max (ms) 90th (ms) 95th (ms) 99th (ms)
1 GET GET /user/search (с Bearer-токеном) 4,077 60.37 0.00 33 18 115 48 54 67

1000 VU



Заключение

Почему такой индекс:

- LOWER(first_name) и LOWER(second_name) повторяют выражения из WHERE;

- text_pattern_ops включает префиксный LIKE 'prefix%'

- Композитный порядок закрывает оба имени и ORDER BY id; id DESC читается в обратном направлении для ASC.

Что дало добавление индекса:

- 1–10 VU: latency ниже в 3–4 раза, RPS почти не меняется — система не упирается в БД.

- 100 VU: seq scan заменён range scan; 25 → 61 RPS, avg 1477 → 33 мс, p95 2397 → 54 мс.

- 1000 VU: 10,6 → 67 RPS, avg 12,5 с → 1,8 с, p99 25,3 с → 4,4 с; остальное ограничивают конкурентность/пул/CPU.

- Поиск переходит из O(N) seq scan в range scan по префиксу, поэтому выигрыш растёт с нагрузкой.

Сводная таблица:

VU	RPS без	RPS с	Avg без, мс	Avg с, мс	p95 без, мс	p95 с, мс	p99 без, мс	p99 с, мс
1	0.56	0.57	102	30	112	43	113	54
10	5.73	6.11	112	28	131	46	168	71
100	24.86	61.24	1 477	33	2 397	54	2 673	67
1000	10.56	67.05	12 555	1 843	21 853	3 397	25 314	4 402

Выигрыш:

VU	RPS	Avg	p95	p99
1	+2%	3.4x	2.6x	2.1x
10	+7%	4.0x	2.8x	2.4x
100	2.5x	44.8x	44.4x	39.9x
1000	6.3x	6.8x	6.4x	5.8x